Contents

Handoff: temperature scaling via Feynman–Kac correctors, and multi-sample (pass@k)

evaluation of CoBit on Sudoku and TinyGSM→GSM8K

Status: paused and handed over. Everything below is reproducible from this branch. **Repo:** `github.com/GBATZOLIS/BitstreamDiffusion` **Branch:** `tasks/fkc-temperature` **Date:** 26 Aug 2026 **Machine:** csic47 (2× A6000); checkpoints also on CSD3.

---

0. Read this first (the five findings)

1. **The paper's GSM8K number reproduces.** 29.42% (388/1319) against the published 29.19%. The pipeline is faithful, so everything downstream is interpretable.
2. **Training is saturated.** The 500k checkpoint gives 29.34% — statistically identical to 425k's 29.42%. "Train longer" is not a lever any more.
3. **Temperature scaling via FKC did not deliver.** With a *healthy* particle population (ESS 10.97/64 at 1024 steps) tempering **significantly hurts**: −5.65 points [−10.62, −0.88]. The apparent gains at 512 steps are mostly plain drift inflation, reproducible with a **single particle** and no Feynman–Kac machinery at all. **This negative is not airtight — see §2.4, which is the main thing to revisit.**
4. **Integration steps are the only significant positive lever we found.** 512→1024 steps gives +7.03 points [+1.95, +12.50] at one sample per problem.
5. **Our pass@k thesis was refuted by our own experiment.** We predicted low-temperature decoding buys discrete models pass@1 by spending the diversity that pass@k needs. It does not: sharpening MDLM lifts pass@1 *and* pass@32 simultaneously, and Duo at T=0.1 beats CoBit at every k.

---

1. Where everything is

What	Path
GSM8K eval (FKC, A1, pass@k, maj@k)	`evaluation/tasks/gsm8k_eval.py`
Sudoku eval (FKC, pass@k)	`evaluation/tasks/sudoku_eval.py`
Shared execution grader	`evaluation/tasks/sandbox_gsm8k.py`
FKC sampler + SMC	`diffusion/continuous/samplers.py`, `diffusion/continuous/smc.py`

What	Path
FKC correctness gates (23 assertions)	`tests/test_fkc.py`
Sweep / probe / shard scripts	`scripts/tasks/*.sh`
CSD3 array jobs + runbook	`scripts/hpc/passk_cobit_csd3.slurm`, `scripts/hpc/PASSK_HPC_README.md`
Baseline (s-flm) patches	`external/s-flm-patches/`
Every FKC cell we ran	`results/fkc_temperature_all_cells.csv`
pass@k raw results	`results/gsm8k_passk_shardA/`
Earlier Sudoku FKC study	`docs/fkc_temperature_findings.md`, `docs/fkc_temperature_report.md`
Workshop plan	`docs/neurips_workshop_plan.pdf`

Baselines live in a separate clone of `github.com/jdeschena/s-flm` (the S-FLM paper's repo, which also ships MDLM/Duo/FLM/CANDI). We modified it; the modifications are vendored here as `external/s-flm-patches/passk_eval.patch` with instructions in that directory.

Environment note: s-flm needs `flash_attn`. Building it cost an hour because pip's `torch` wheel was CUDA 13 while local `nvcc` is 12.1. The working recipe on csic47 was to **clone the existing `pytorch` conda env** (torch 2.5.1 / cu121 / flash_attn 2.5.9) and add s-flm's deps — see §5. Their README says they develop in the NGC PyTorch container, which is the better route on CSD3.

---

2. Part I — Temperature scaling via Feynman–Kac correctors

2.1 What was implemented

The goal was a *principled* low-temperature analogue for CoBit: sample from the tempered target p^β using a Feynman–Kac corrector (Skreta et al.), rather than the heuristic categorical temperature that MDLM/Duo use. Two mechanisms exist in the code:

- **FKC tempering (`--sampler_kind fkc_em`).** K weighted particles per prompt targeting p^β. β enters in *two* places, which matters for interpreting everything below:
  1. the **proposal drift is scaled by β** — `samplers.py:2955`, `x = x + h * beta * d`;
  2. the **importance weight** $\frac{1}{2}\beta(\beta-1)\Delta\sigma^2\|s\|^2$, which drives resampling.
- **Track A1 score-temperature (`--score_temp_tau`).** Particle-free, zero extra NFE. Rescales the PF-ODE score by $\kappa(\sigma)=(v+\sigma^2)/(\tau v+\sigma^2)$: no effect at high σ, $\to 1/\tau$ as $\sigma\to 0$.

Both are wired into Sudoku and (new, this work) GSM8K. All 23 FKC gates pass on this branch, including the two churn-interval regressions.

2.2 What we ran

37 cells, all in `results/fkc_temperature_all_cells.csv`. The load-bearing ones:

(a) β at 1024 steps with a healthy population — the decisive test.

cell	ESS	pass@1-equivalent
β=1.05, K=64, 1024 steps	**10.97 / 64**	29.5%
β=1.0 control (K=1, 1024 steps, same problems)	–	**35.2%**
paired difference		**−5.65 [−10.62, −0.88]**

This is the only configuration where the corrector ran with a genuinely healthy cloud, and tempering **significantly hurt**.

(b) β at 512 steps — a real trend, but not from FKC. Across β ∈ [1.01, 1.08] at K=32 the trend is significant (slope 95% CI [0.085, 0.705], p(slope≤0)=0.0066, ≈+2.7 points over the range). But:

configuration	best accuracy
K=32 corrected FKC, β=1.15	34.0%
K=1, drift only (no weights, no resampling), β=1.20	**32.8%**
β=1.0 control	28.5%

So ~+4.3 of the ~+5.5 total is obtainable with **one particle** and no Feynman–Kac at all; the corrector adds ≈+1.2 (β=1.05) to +1.9 (β=1.08), neither individually significant.

(c) K-scaling does nothing. K=64 vs K=32 at β=1.05, 512 steps: −0.06 [−2.21, +2.04].

(d) Diversity collapses as β rises. `mean_distinct_answers` falls 6.17 → 0.95 and ESS 2.12 → 1.00 over β ∈ [1.01, 1.20]; at β≥1.15 all 32 particles return the same answer, so the "32-particle run" is a 1-particle run in disguise.

(e) A1 score-temperature fails too. At 1024 steps, n=256: τ=1.0 → 34.0%, τ=0.7 → 34.8%, τ=0.5 → 32.0%, **τ=0.3 → 0.0%** (total collapse — no valid programs). It does *not* transfer from Sudoku, where the same knob gave +2.2 monotone with no cliff.

(f) Step count dominates everything. 512→1024 at K=1: **+7.03 [+1.95, +12.50]**. The best 512-step tempered configuration (34.0%) still loses to plain 1024-step single-sample (35.5%).

2.3 Our reading

β behaves like a **step-size correction, not a temperature**. It inflates the whole PF-ODE drift; at 512 steps the integrator undershoots and the inflation compensates, and at 1024 steps the same inflation overshoots and hurts. That explains the sign flip with step count, why the optimum drifts upward as the schedule coarsens, and why K makes no difference.

2.4 Why the student should not accept this negative (the important section)

Eight concrete reasons the result may be an artefact of *our* implementation or application:

1. **The diagnostic that would explain it is structurally blind.** `logw` is correctly reset at every resampling event (standard SMC — the weights are consumed by resampling). But `log_weights_final` is captured right after the last reset, and at β≥1.04 resampling fires at essentially every step, so the recorded weights are **exactly uniform** (0.03125000 = 1/32 to eight decimals). We therefore *cannot measure* whether the FKC weight ranks correctness in the arm where the gain lives. **Fix first:** accumulate a non-reset `cumulative_path_potential` gathered along ancestry — purely diagnostic, cannot change results. This is audit item #6. Where weights *do* survive a full path (naive arm), a real signal appears: w(correct)/w(incorrect) = **1.0706** at β=1.08, growing with β as ½β(β−1) predicts.
2. **K may be orders of magnitude too small.** The log-weight is *extensive in free bits*: GSM8K has ~7168 (vs Sudoku's 356). SMC needs K to grow roughly like exp(Var log w). We never exceeded K=64. A serious test might need K in the hundreds or thousands, which is a distributed-sampling engineering problem, not a hyperparameter sweep.
3. **The proposal is only leading-order.** `edm_churn` with β>1 is an approximation — tempering does not commute with Gaussian churn (`_warn_churn_fkc_inexact`). Halving the step count makes it worse, and we never ran the $\gamma_i = S\_churn/N$ refinement study the warning asks for.
4. **The hyperparameter space is barely explored.** Fixed throughout: `ess_threshold=0.5`, `resampling_policy=ess`, `sc_policy=inherit`, default `prior_mode`. Never tried on GSM8K: `every_step_active`, `sc_policy ∈ {zero, stateless_two_pass}`, `resample_entropy_frac`, or `prior_mode=forward_marginal_diag`.

5. **β, γ and step count are entangled and were never gridded.** γ was pinned at 0.41 (the EDM cap) in every cell. Effective Langevin strength is λ ≈ S_churn·π(log σ) with S_churn = γ(N−1), so changing N changes stochasticity *and* resolution together. A proper (β, γ, N) grid has not been run.
6. **The corrected-vs-naive A/B is thin.** Two β values, one step count, n=256. That is the experiment that isolates "does the corrector earn its keep", and it deserves more.
7. **Only constant β.** Annealed or σ-dependent β schedules were never tried, though the entropy-rate machinery to define one already exists.
8. **CFG-FKC was never tested on GSM8K.** The Prop 3.1 two-model variant needs a conditioning-dropout checkpoint; one exists on CSD3 (`runs/tasks/tinygsm/cobit_raw_binary_bits_cfg`) and was never used here.

Also unfixed: audit items **#7–#10** (`final_unique_ancestors` mis-computed; per-batch seed reset correlates noise; Sudoku validity check ignores clue-consistency; pass@K over one SMC cloud is genealogically correlated, unlike K independent draws). See `docs/fkc_temperature_findings.md`.

---

3. Part II — Multi-sample (pass@k) evaluation

3.1 Why

No prior work in this comparison reports multi-sample metrics. We read S-FLM (arXiv:2605.11125) in full: Sudoku is scored with **one sample per puzzle**, GSM8K with **one generated solution per problem**, and pass@k / majority vote / best-of-n appear nowhere. Since S-FLM is the source of every baseline number in the CoBit draft, this is an open lane. pass@k also bounds what RL post-training can realise (Yue et al., arXiv:2504.13837), and diffusion-LM post-training so far is masked-only (d1 / diffu-GRPO).

3.2 Protocol (identical for every method)

K=32 samples/problem · 1024 steps · fp32 · the **same** 256 random test problems (seed 0, `external/s-flm-patches/data_gsm8k_shard_manifest.json`) · the **same** execution grader (their `sandbox_gsm8k.py` was byte-identical to ours apart from our `predict_answer` refactor, so we installed ours in both) · unbiased Codex estimator pass@k = 1 − C(n−c,k)/C(n,k), so one K=32 run yields the whole k=1…32 curve · maj@k votes over **executed answers**, not program text.

3.3 Results (shard A, 256 problems)

method	pass@1	pass@2	pass@4	pass@8	pass@16	**pass@32**	maj@32
CoBit (no tempera-ture)	29.1%	38.3%	46.4%	53.5%	59.9%	66.0%	45.7%
Duo T=0.1	**37.1%**	**50.2%**	**61.4%**	**70.2%**	**77.5%**	**83.6%**	**61.3%**
MDLM T=0.1	35.4%	48.1%	59.1%	67.7%	74.7%	80.9%	59.4%
MDLM T=1.0	16.1%	26.3%	38.4%	49.9%	59.7%	68.0%	52.3%

The thesis is refuted. Low-temperature decoding does not spend headroom to buy pass@1: sharpening MDLM lifts pass@1 (16.1→35.4%) *and* pass@32 (68.0→80.9%). The trade exists only in *relative* terms — the k=1→32 multiplier falls 4.2× → 2.3× — but the absolute curve rises everywhere. Duo at T=0.1 dominates CoBit at every k.

CoBit's own headroom is real (29.1 → 66.0%, 2.3×, with ESS=32/32, i.e. 32 genuinely independent churn samples), just not competitive on GSM8K. On **Sudoku** CoBit remains the best model overall (98.5 / 91.7 / 65.9 single-sample vs Duo 96.3 / 84.7 / 58.4), and the earlier FKC study measured hard-Sudoku maj@16 ≈ 91% and pass@16 ≈ 94% — worth re-measuring properly.

3.4 What is incomplete

- `duo_T1.0` **crashed**: `OverflowError: int too large to convert to float` in their metrics path (not OOM). Needed to complete Duo's temperature contrast.
- **S-FLM cell never ran.** Runner exists (`external/s-flm-patches/passk_sfm.sh`, sphere-arch config family, top-1 velocity = their best GSM8K variant) but is un-smoke-tested.
- **Shard B (the other 1063 problems) not run.** CSD3 array jobs are written and pushed. Shards are disjoint by construction, so merging is concatenation — `merge_passk.py`.
- **Sudoku pass@k** started and was killed to free GPUs; baselines would need retraining (no released Sudoku checkpoints, but their paper says <2 h each on one L40S).

---

4. Part III — Traps that cost us time (do not rediscover these)

1. **`sigma_data` is not in the TinyGSM checkpoints.** They predate the commit that persists it. Pass `--sigma_data 0.399844765663147` explicitly. The config default is **0.5**, it is wrong, and it fails silently.
2. **Never use a test-set prefix as a subset.** The first 256 problems run ~6 points hot (35.55% vs 29.42% at identical config). Use the seed-0 random shard.
3. **The GSM8K result tag contains no checkpoint step**, and `out_dir` defaults to `<run>/gsm8k_eval/`. Re-running a different checkpoint with default paths **overwrites** the previous artifact. Always pass `--out_dir`.
4. **Compare paired, not aggregate.** Per-prompt vectors are now stored (`per_prompt_particle_acc`); between-prompt variance dominates and cancels only in the paired difference.
5. **Duo/MDLM default to fp64** (`sampler.use_float64: true`). fp32 is 2.75× faster; we measured Duo T=1 fp32 at 19.5% vs 17.2% published, 95% CIs overlapping ([17.4, 21.6] vs [15.2, 19.2]), and used fp32 throughout. **This must be stated in any write-up.**
6. **The paper's 29.19% belongs to the main-run 425k checkpoint**, not the Isambard one (which gives 28.96%). The artifact was simply never saved.

---

5. How to run things

```
# CoBit: pass@k on GSM8K (K independent churn samples; beta=1, no resampling)
python -m evaluation.tasks.gsm8k_eval \
  --config configs/tasks/tinygsm_bits.py \
  --checkpoint runs/tasks/tinygsm/cobit_raw_binary_bits/checkpoints/step=000425000.pt \
  --sampler_kind fkc_em --proposal edm_churn --churn_gamma 0.41 \
  --beta 1.0 --num_particles 32 --resampling_policy never --final_resample 0 \
  --steps 1024 --limit 1319 --batch_size 2 --ema 1 --seed 42 \
  --sigma_data 0.399844765663147 --out_dir <somewhere>

# CoBit: FKC tempering (beta > 1 turns the corrector on)
#   ... --beta 1.05 --resampling_policy ess --ess_threshold 0.5 --final_resample 1

# CoBit: A1 score-temperature (particle-free, 1x NFE)
#   ... --score_temp_tau 0.7

# FKC gates (must stay green)
```

```
PYTHONPATH=. python tests/test_fkc.py

# Baselines: see external/s-flm-patches/README.md
```

---

6. Suggested order of work

1. **Fix audit #6** (non-reset cumulative path potential). Without it the corrected arm cannot be diagnosed, and every FKC conclusion rests on an unmeasurable quantity. Cheap, diagnostic only.
2. **Finish the pass@k table**: fix `duo_T1.0`, run S-FLM, run shard B on CSD3, merge to the full 1319. This is the part closest to publishable.
3. **Sudoku multi-sample**, where CoBit is actually strongest — CoBit pass@k on all three difficulties, and retrain the Duo/MDLM Sudoku baselines (cheap) for a like-for-like table.
4. **Then** revisit FKC properly, with §2.4 as the checklist: larger K first, then the (β, γ, N) grid, then proposal accuracy, then the untried policies.